شرح سطر بسطر ١٥ — إعدادات حسابي: التفضيلات والودجتات والأوراق

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

هذه القطع تقف خلف مجموعات «الدراسة» و«التنزيل والتخزين» و«عام» في شاشة حسابي: مستودع التفضيلات (الباك اند)، وودجتات الصفوف المشتركة، وأوراق الاختيار والمعلومات، وشاشة إدارة التنزيلات.

التفضيلات (حجم الخط، أصوات الألعاب، Wi-Fi فقط، تنزيل الفيديو تلقائيًا) محفوظة لكل طالب في جدول `app_settings` الموجود بمفاتيح `pref.<id>.<name>`: لا جدول جديد ولا ترحيل للقاعدة.

ما له أثر فعلي الآن: الصف (يغيّر الملف في القاعدة) وحجم الخط (يغيّر كل التطبيق). أما أصوات الألعاب وWi-Fi وتنزيل الفيديو فتُحفظ وتنتظر ميزاتها، وتقول الواجهة ذلك. «الوضع الليلي» معطّل مع «قريبًا».

lib/features/account/preferences_repository.dart — تفضيلات الطالب (الباك اند)

`lib/features/account/preferences_repository.dart`

الأسطر 2–1

```
import '../../core/db/app_database.dart';
import '../../core/db/daos/settings_dao.dart';
```

ماذا يفعل؟ استيراد القاعدة و`SettingsDao`.

لماذا بهذه الطريقة؟ للوصول إلى `settingsDao` وإلى `SettingsDao.profilePrefsPrefix` (مصدر البادئة الوحيد).

الأسطر 8–4

```
/// حجم الخط الذي يختاره الطالب. `scale` يُضرب في مقياس النص في كل التطبيق.
enum FontSizeChoice {
  small(0.9),
  medium(1.0),
  large(1.15);
```

ماذا يفعل؟ تعداد `FontSizeChoice` بثلاث قيم يحمل كل منها معامل التكبير: 0.9 و1.0 و1.15.

لماذا بهذه الطريقة؟ تعداد محسّن (enhanced enum) يربط الاسم بالرقم في مكان واحد، فلا تتناثر الأرقام. الفرق بين الأحجام معتدل ليبقى التصميم سليمًا.

الأسطر 13–10

```
  const FontSizeChoice(this.scale);

  final double scale;
}
```

ماذا يفعل؟ باني ثابت وحقل `scale`.

لماذا بهذه الطريقة؟ `const` ليُستعمل في قوائم ثابتة.

الأسطر 15–22

```
/// تفضيلات طالب واحد. القيم الافتراضية هي ما يراه ملف جديد.
class ProfilePrefs {
  const ProfilePrefs({
    this.fontSize = FontSizeChoice.medium,
    this.gameSounds = true,
    this.wifiOnly = true,
    this.autoVideo = false,
  });
```

ماذا يفعل؟ صنف `ProfilePrefs` غير قابل للتغيير بأربعة تفضيلات وقيم افتراضية: خط متوسط، أصوات مفعّلة، Wi-Fi فقط مفعّل، تنزيل الفيديو تلقائيًا مطفأ.

لماذا بهذه الطريقة؟ الافتراضيات هي ما يراه ملف جديد. الاثنان الأخيران محافظان: لا نستهلك باقة الطالب دون إذنه.

الأسطر 24–26

```
  /// يقرأ التفضيلات من خريطة (اسم التفضيل ← نص). أي قيمة ناقصة أو غير مفهومة
  /// ترجع إلى الافتراضي بدل أن تُفشل الشاشة.
  factory ProfilePrefs.fromMap(Map<String, String> raw) {
```

ماذا يفعل؟ مصنّع `fromMap` يبني التفضيلات من خريطة (اسم ← نص).

لماذا بهذه الطريقة؟ القاعدة تخزّن نصوصًا فقط (مفتاح/قيمة)، فنحوّلها هنا إلى أنواع.

الأسطر 27–31

```
    bool flag(String key, bool fallback) => switch (raw[key]) {
      'true' => true,
      'false' => false,
      _ => fallback,
    };
```

ماذا يفعل؟ دالة محلية `flag` : `"true"` ← true، `"false"` ← false، وأي شيء آخر (أو غائب) ← الافتراضي.

لماذا بهذه الطريقة؟ تعبير `switch` يعطي سلوكًا آمنًا: قيمة تالفة لا تُفشل الشاشة.

الأسطر 33–38

```
    const defaults = ProfilePrefs();
    return ProfilePrefs(
      fontSize: FontSizeChoice.values.firstWhere(
        (c) => c.name == raw[PreferencesRepository.fontSizeKey],
        orElse: () => defaults.fontSize,
      ),
```

ماذا يفعل؟ `defaults` ثابتة، ثم نبحث عن حجم الخط باسمه (`c.name`) وإلا الافتراضي.

لماذا بهذه الطريقة؟ `firstWhere` مع `orElse` بدل `byName` الذي يرمي استثناء: قيمة مجهولة (مثلًا من إصدار قديم) لا تكسر التطبيق.

الأسطر 39–46

```
      gameSounds: flag(
        PreferencesRepository.gameSoundsKey,
        defaults.gameSounds,
      ),
      wifiOnly: flag(PreferencesRepository.wifiOnlyKey, defaults.wifiOnly),
      autoVideo: flag(PreferencesRepository.autoVideoKey, defaults.autoVideo),
    );
  }
```

ماذا يفعل؟ بقية التفضيلات عبر `flag` بمفاتيحها الثابتة، ثم إغلاق المصنّع.

لماذا بهذه الطريقة؟ المفاتيح ثوابت في المستودع فلا تُكتب النصوص في مكانين.

الأسطر 48–58

```
  final FontSizeChoice fontSize;

  /// أصوات الألعاب. يُحفظ الآن ويُقرأ حين تُبنى الألعاب.
  final bool gameSounds;

  /// التنزيل عبر Wi-Fi فقط. يقرؤه مدير التنزيلات لاحقًا.
  final bool wifiOnly;

  /// تنزيل الفيديو تلقائيًا. يقرؤه مدير التنزيلات لاحقًا.
  final bool autoVideo;
}
```

ماذا يفعل؟ الحقول الأربعة، وعلى كل منها تعليق يقول من سيقرؤه.

لماذا بهذه الطريقة؟ الأصوات والتنزيل «تُحفظ الآن وتُقرأ لاحقًا»: مكتوب في الكود نفسه كي لا يظن مطوّر أنها تعمل.

الأسطر 60–63

```
/// تفضيلات كل طالب، محفوظة في جدول `app_settings` بمفاتيح `pref.<id>.<name>`.
/// لا نحتاج جدولًا جديدًا ولا ترحيلًا للقاعدة.
class PreferencesRepository {
  PreferencesRepository(this._db);
```

ماذا يفعل؟ تعليق وصنف `PreferencesRepository` وباني بالقاعدة.

لماذا بهذه الطريقة؟ التعليق يشرح أن لا جدول جديد ولا ترحيل: نستعمل `app_settings` بمفاتيح `pref.<id>.<name>`.

السطر 65

```
  final AppDatabase _db;
```

ماذا يفعل؟ حقل القاعدة.

لماذا بهذه الطريقة؟ نص ثابت للوصول.

الأسطر 67–70

```
67    static const fontSizeKey = 'fontSize';
68    static const gameSoundsKey = 'gameSounds';
69    static const wifiOnlyKey = 'wifiOnly';
70    static const autoVideoKey = 'autoVideo';
```

ماذا يفعل؟ ثوابت أسماء المفاتيح الأربعة.

لماذا بهذه الطريقة؟ تستعملها `fromMap` والكتابة معًا فلا تختلف.

الأسطر 72–73

```
72    static String _prefix(String profileId) =>
73        SettingsDao.profilePrefsPrefix(profileId);
```

ماذا يفعل؟ `_prefix` : يستدعي `SettingsDao.profilePrefsPrefix` (`pref.<معرّف الطالب>.`).

لماذا بهذه الطريقة؟ النقطة في النهاية تمنع تداخل معرّفين أحدهما بادئة للآخر (`pref.a.` لا تطابق `pref.ab.x`). اختبار يغطي ذلك.

الأسطر 75–85

```
75    Stream<ProfilePrefs> watch(String profileId) {
76      final prefix = _prefix(profileId);
77      return _db.settingsDao
78          .watchByPrefix(prefix)
79          .map(
80            (rows) => ProfilePrefs.fromMap({
81              for (final e in rows.entries)
82                e.key.substring(prefix.length): e.value,
83            }),
84          );
85    }
```

ماذا يفعل؟ `watch(profileId)` : بث بتفضيلات الطالب. نجلب كل المفاتيح ببادئته، ونحذف البادئة من كل مفتاح، ونمرّر الخريطة إلى `fromMap` .

لماذا بهذه الطريقة؟ بث لا قراءة لمرة واحدة: فور تغيير أي تفضيل تتحدّث الشاشة وحجم الخط في كل التطبيق دون إعادة فتح.

السطر 87

```
87    Future<ProfilePrefs> read(String profileId) => watch(profileId).first;
```

ماذا يفعل؟ `read` : أول قيمة من البث.

لماذا بهذه الطريقة؟ للاختبارات ولمن يحتاج قراءة لمرة واحدة.

الأسطر 89–90

```
89    // مفتاح لكل درس): علامة الصفحة في شاشة الدرس) الدروس المحفوظة.
90    static const _bookmarkPart = 'bookmark.';
```

ماذا يفعل؟ الدروس المحفوظة: جزء المفتاح الثابت `bookmark.` تحت مفاتيح الطالب.

لماذا بهذه الطريقة؟ علامة الصفحة في شاشة الدرس تحتاج حفظًا لكل طالب، ونفس جدول الإعدادات يكفي بلا جدول جديد.

الأسطر 92–98

```
  /// معرّفات الدروس التي حفظها الطالب.
  Stream<Set<String>> watchBookmarks(String profileId) {
    final prefix = '${_prefix(profileId)}$_bookmarkPart';
    return _db.settingsDao
        .watchByPrefix(prefix)
        .map((rows) => {for (final k in rows.keys) k.substring(prefix.length)});
  }
```

ماذا يفعل؟ `watchBookmarks`: بث بمعرّفات الدروس المحفوظة، بقصّ البادئة من كل مفتاح.

لماذا بهذه الطريقة؟ وجود المفتاح هو المعلومة؛ القيمة لا تهم.

الأسطر 100–111

```
  Future<void> setBookmarked(
    String profileId,
    String lessonId,
    bool saved,
  ) async {
    final key = '${_prefix(profileId)}$_bookmarkPart$lessonId';
    if (saved) {
      await _db.settingsDao.write(key, 'true');
    } else {
      await _db.settingsDao.remove(key);
    }
  }
```

ماذا يفعل؟ `setBookmarked`: يكتب المفتاح عند الحفظ ويحذفه عند الإلغاء.

لماذا بهذه الطريقة؟ الحذف بدل كتابة "false" يبقي الجدول نظيفًا ويجعل البث يعكس المحفوظ فقط.

الأسطر 113–114

```
  Future<void> setFontSize(String profileId, FontSizeChoice choice) =>
      _write(profileId, fontSizeKey, choice.name);
```

ماذا يفعل؟ `setFontSize`: يكتب اسم القيمة (`small/medium/large`).

لماذا بهذه الطريقة؟ الاسم أوضح وأثبت من الرقم الترتيبي إن تغيّر ترتيب التعداد.

الأسطر 116–123

```
  Future<void> setGameSounds(String profileId, bool value) =>
      _write(profileId, gameSoundsKey, '$value');

  Future<void> setWifiOnly(String profileId, bool value) =>
      _write(profileId, wifiOnlyKey, '$value');

  Future<void> setAutoVideo(String profileId, bool value) =>
      _write(profileId, autoVideoKey, '$value');
```

ماذا يفعل؟ `setGameSounds` و`setWifiOnly` و`setAutoVideo`: تكتب القيمة المنطقية نصًّا.

لماذا بهذه الطريقة؟ جدول الإعدادات نصّي؛ `flag` في `fromMap` يعيدها منطقية.

الأسطر 125–128

```
  Future<void> _write(String profileId, String name, String value) {
    return _db.settingsDao.write('${_prefix(profileId)}$name', value);
  }
}
```

ماذا يفعل؟ `_write` : يكتب المفتاح الكامل `pref.<id>.<name>` بالقيمة، والإغلاق.

لماذا بهذه الطريقة؟ كل الكتابات تمرّ من مكان واحد فلا يُخطأ في بناء المفتاح.

lib/features/account/widgets/setting_widgets.dart — بطاقة المجموعة وصفوف الإعدادات

lib/features/account/widgets/setting_widgets.dart

تغييران للسلاسة والحذف: (1) `labelColor` (افتراضيًا `ink`) ليظهر عنوان صف «حذف الملف» مرجانيًا؛ (2) قيمة الصف (مثل «الخامس» أو «متوسط») داخل `AnimatedSwitcher` بمدة 180 ملّي ثانية ومفتاح `ValueKey(value)` فتتبدّل بتلاشٍ بدل قفزة عند تغييرها.

الأسطر 1–2

```
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';
```

ماذا يفعل؟ استيراد Material وflutter_svg (لسهم الصف).

لماذا بهذه الطريقة؟ السهم أصل من Figma.

الأسطر 4–10

```
import '../../../core/constants/app_assets.dart';
import '../../../core/strings/ar.dart';
import '../../../core/theme/app_colors.dart';
import '../../../core/theme/app_text_styles.dart';
import '../../../core/widgets/icon_tile.dart';
import '../../../core/widgets/rizma_card.dart';
import '../../../core/widgets/rizma_switch.dart';
```

ماذا يفعل؟ استيراد مسارات الأصول والنصوص والألوان والأنماط ومربّع الأيقونة والبطاقة والمفتاح.

لماذا بهذه الطريقة؟ مكوّنات مشتركة موجودة سلفًا؛ لا نكرر رسمها.

الأسطر 12–23

```dart
/// بطاقة مجموعة إعدادات: عنوان ثم صفوف، وسطر ملاحظة اختياري في أسفلها.
class SettingsGroupCard extends StatelessWidget {
  const SettingsGroupCard({
    super.key,
    required this.title,
    required this.children,
    this.footnote,
  });

  final String title;
  final List<Widget> children;
  final String? footnote;
```

ماذا يفعل؟ تعليق وصنف `SettingsGroupCard` وحقوله: العنوان والصفوف وملاحظة اختيارية.

لماذا بهذه الطريقة؟ الودجت نفسه لكل المجموعات الأربع فيتطابق شكلها.

الأسطر 25–31

```dart
  @override
  Widget build(BuildContext context) {
    return RizmaCard(
      padding: const EdgeInsetsDirectional.fromSTEB(16, 14, 16, 4),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
```

ماذا يفعل؟ `RizmaCard` : `build` بحشوة 16/14/16/4 و `Column` ممتد.

لماذا بهذه الطريقة؟ الحشوة السفلية 4 لأن لكل صف خطًا سفليًا يضيف فراغًا.

الأسطر 32–36

```dart
          Padding(
            padding: const EdgeInsets.only(bottom: 4),
            child: Text(title, style: AppTextStyles.section18),
          ),
          ...children,
```

ماذا يفعل؟ عنوان المجموعة ثم الصفوف (`...children`).

لماذا بهذه الطريقة؟ نمط `...` . `section18` تفرد القائمة داخل `children` .

الأسطر 37–41

```dart
          if (footnote != null)
            Padding(
              padding: const EdgeInsets.symmetric(vertical: 10),
              child: Text(footnote!, style: AppTextStyles.caption13),
            ),
```

ماذا يفعل؟ إن وُجدت ملاحظة نعرضها بنمط `caption13` بحشوة رأسية 10.

لماذا بهذه الطريقة؟ `if` داخل القائمة بلا ودجت إضافي. تُستعمل في «يعمل مع التنزيلات عند توفرها».

الأسطر 42–46

```
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

الأسطر 48–52

```
/// إعداد: صف. أيقونة ملوّنة، عنوان، ثم قيمة أو مفتاح أو سهم، وخط سفلي رفيع.
///
/// - [onTap] زر: الصف كله غير فارغ (ينتهي بسهم ما لم يوجد [trailing]).
/// - [trailing]: بدل السهم (مثل مفتاح تشغيل) في الطرف. عنصر.
/// - [onTap] (مثل صف فيه مفتاح، أو معطّل «قريبًا») قابل للضغط غير صف: فارغ.
```

ماذا يفعل؟ تعليق `SettingRow`: متى يكون الصف زرًّا ومتى لا.

لماذا بهذه الطريقة؟ ثلاث حالات في ودجت واحد: صف قابل للضغط، صف بعنصر طرفي (مفتاح)، وصف غير قابل للضغط (معطّل).

الأسطر 53–70

```
class SettingRow extends StatelessWidget {
  const SettingRow({
    super.key,
    required this.icon,
    required this.iconColor,
    required this.tint,
    required this.label,
    this.onTap,
    this.value,
    this.valueColor = AppColors.teal,
    this.valueLtr = false,
    this.labelColor = AppColors.ink,
    this.trailing,
    this.semanticsLabel,
  });

  /// اسم أيقونة من مجموعة التصميم (`assets/icons/ui`).
  final String icon;
```

ماذا يفعل؟ صنف `SettingRow` وباني بمعاملات: اسم الأيقونة (`String` من مجموعة التصميم) وألوانها والعنوان، واختيارية: الضغط والقيمة ولونها واتجاهها والعنصر الطرفي ووصف القارئ.

لماذا بهذه الطريقة؟ أغلبها اختياري لأن كل صف يحتاج جزءًا منها فقط. **تغيّر:** الأيقونة صارت اسمًا من `assets/icons/ui` بدل `IconData` من Material، فتشبه أيقونات التصميم وتظهر في لقطات الشاشة.

الأسطر 71–82

```
71    final Color iconColor;
72    final Color tint;
73    final String label;
74    final String? value;
75    final Color valueColor;
76    final bool valueLtr;
77    final Color labelColor;
78    final VoidCallback? onTap;
79    final Widget? trailing;
80
81    /// وصف قارئ الشاشة. الافتراضي «العنوان، القيمة».
82    final String? semanticsLabel;
```

ماذا يفعل؟ الحقول وتعليق وصف قارئ الشاشة.

لماذا بهذه الطريقة؟ الافتراضي «العنوان، القيمة» فيُقرأ الصف جملة واحدة.

الأسطر 84–86

```
84    @override
85    Widget build(BuildContext context) {
86      final tappable = onTap != null;
```

ماذا يفعل؟ `build` ونحدد هل الصف قابل للضغط (`onTap != null`).

لماذا بهذه الطريقة؟ منطق السهم والـ`InkWell` كله مبني على هذا المتغيّر.

الأسطر 88–92

```
88      final content = Container(
89        constraints: const BoxConstraints(minHeight: 56),
90        decoration: const BoxDecoration(
91          border: Border(bottom: BorderSide(color: AppColors.line)),
92        ),
```

ماذا يفعل؟ حاوية بحد أدنى للارتفاع 56 وخط سفلي رفيع.

لماذا بهذه الطريقة؟ 56 كما في التصميم؛ الخط يفصل الصفوف.

الأسطر 93–95

```
93        child: Row(
94          spacing: 12,
95          children: [
```

ماذا يفعل؟ `Row` بمسافة 12: أول عنصر على اليمين.

لماذا بهذه الطريقة؟ RTL: الأيقونة أولًا ثم العنوان.

الأسطر 96–103

```
        IconTile(
          iconName: icon,
          iconColor: iconColor,
          color: tint,
          size: 40,
          iconSize: 20,
          radius: 12,
        ),
```

ماذا يفعل؟ مربّع الأيقونة 40 بـ `iconName`.

لماذا بهذه الطريقة؟ نفس مقاسات باقي الشاشات، ومن مجموعة أيقونات التصميم.

الأسطر 104–109

```
        Expanded(
          child: Text(
            label,
            style: AppTextStyles.body15.copyWith(color: labelColor),
          ),
        ),
```

ماذا يفعل؟ العنوان في `Expanded`.

لماذا بهذه الطريقة؟ يدفع القيمة والسهم إلى اليسار.

الأسطر 110–120

```
        if (value != null)
          // تبدّل القيمة (مثل الصف أو حجم الخط) بتلاشٍ بدل قفزة.
          AnimatedSwitcher(
            duration: const Duration(milliseconds: 180),
            child: Text(
              value!,
              key: ValueKey(value),
              textDirection: valueLtr ? TextDirection.ltr : null,
              style: AppTextStyles.label14.copyWith(color: valueColor),
            ),
          ),
```

ماذا يفعل؟ إن وُجدت قيمة نعرضها بلونها، وباتجاه LTR إن لزم.

لماذا بهذه الطريقة؟ اتجاه LTR للرمز `RZ-XXXX`.

الأسطر 121–126

```
        if (trailing != null)
          trailing!
        else if (tappable)
          SvgPicture.asset(AppAssets.fwdRow, width: 18, height: 18),
      ],
    ),
```

ماذا يفعل؟ العنصر الطرفي إن وُجد، وإلا سهم الصف إن كان قابلًا للضغط، ثم إغلاق `Row` والحاوية.

لماذا بهذه الطريقة؟ الصف بمفتاح لا يحتاج سهمًا، والصف المعطّل لا يحتاج شيئًا.

السطر 127

```
    );
```

ماذا يفعل؟ إغلاق تعريف `content`.

لماذا بهذه الطريقة؟ نبني المحتوى مرة ونغلّفه بحسب الحالة.

السطر 129

```
    if (!tappable) return content;
```

ماذا يفعل؟ إن لم يكن قابلًا للضغط نعيد المحتوى كما هو.

لماذا بهذه الطريقة؟ صف المفتاح لا يكون كله زرًّا: المفتاح وحده يستجيب (كما في التصميم).

الأسطر 131–137

```
    return Semantics(
      button: true,
      label: semanticsLabel ?? (value == null ? label : '$label، $value'),
      excludeSemantics: true,
      child: InkWell(onTap: onTap, child: content),
    );
  }
```

ماذا يفعل؟ وإلا `Semantics` زر بوصف «العنوان، القيمة» ويستبعد دلالات الأبناء، و`InkWell` حول المحتوى.

لماذا بهذه الطريقة؟ قارئ الشاشة يقرأ «الرمز السري، مفعّل» جملة واحدة بدل عناصر منفصلة.

السطر 138

```
}
```

ماذا يفعل؟ إغلاق الصنف.

لماذا بهذه الطريقة؟ نهاية `SettingRow`.

الأسطر 141–140

```
/// صف فيه مفتاح تشغيل. المفتاح وحده هو القابل للضغط (كما في التصميم)،
/// وهو يحمل وصف قارئ الشاشة بنفسه.
```

ماذا يفعل؟ تعليق `SettingSwitchRow`.

لماذا بهذه الطريقة؟ المفتاح وحده القابل للضغط ويحمل وصف القارئ.

الأسطر 142–154

```
class SettingSwitchRow extends StatelessWidget {
  const SettingSwitchRow({
    super.key,
    required this.icon,
    required this.iconColor,
    required this.tint,
    required this.label,
    required this.value,
    required this.onChanged,
  });

  /// اسم أيقونة من مجموعة التصميم (`assets/icons/ui`).
  final String icon;
```

ماذا يفعل؟ صنف `SettingSwitchRow` وباني بمعاملات الصف (الأيقونة اسمًا) والقيمة ودالة التغيير.

لماذا بهذه الطريقة؟ يغلّف `SettingRow` فلا نكرر رسم الصف.

الأسطر 155–159

```
  final Color iconColor;
  final Color tint;
  final String label;
  final bool value;
  final ValueChanged<bool> onChanged;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 161–175

```
  @override
  Widget build(BuildContext context) {
    return SettingRow(
      icon: icon,
      iconColor: iconColor,
      tint: tint,
      label: label,
      trailing: RizmaSwitch(
        value: value,
        onChanged: onChanged,
        semanticsLabel: label,
      ),
    );
  }
}
```

ماذا يفعل؟ `build`: يعيد `SettingRow` وعنصره الطرفي `RizmaSwitch` بالقيمة ودالة التغيير ووصف القارئ = العنوان.

لماذا بهذه الطريقة؟ `RizmaSwitch` موجود سلفًا (52×32 بحسب التصميم). وصف القارئ هو العنوان فيُقرأ «أصوات الألعاب، مفعّل».

الأسطر 178–177

```
177 /// «صف معطّل لميزة لم تُبنَ بعد: يظهر بشفافية مع كلمة «قريبًا
178 /// ومفتاح مطفأ لا يستجيب.
```

ماذا يفعل؟ تعليق `SoonSettingRow`.

لماذا بهذه الطريقة؟ صف لميزة لم تُبنَ: ظاهر لكن لا يستجيب.

الأسطر 192–179

```
179 class SoonSettingRow extends StatelessWidget {
180   const SoonSettingRow({
181     super.key,
182     required this.icon,
183     required this.iconColor,
184     required this.tint,
185     required this.label,
186   });
187 
188   /// اسم أيقونة من مجموعة التصميم (`assets/icons/ui`).
189   final String icon;
190   final Color iconColor;
191   final Color tint;
192   final String label;
```

ماذا يفعل؟ صنف `SoonSettingRow` وباني وحقوله (اسم الأيقونة وألوانها والعنوان).

لماذا بهذه الطريقة؟ بلا قيمة ولا دالة: لا شيء يمكن أن يتغيّر.

الأسطر 199–194

```
194   @override
195   Widget build(BuildContext context) {
196     return Semantics(
197       enabled: false,
198       label: '$label، ${ArStrings.accountSoon}',
199       excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` في `build`: معطّل (`enabled: false`) بوصف «العنوان، قريبًا».

لماذا بهذه الطريقة؟ قارئ الشاشة يعرف أن الصف معطّل ولماذا.

الأسطر 202–200

```
200       child: Opacity(
201         opacity: 0.55,
202         child: IgnorePointer(
```

ماذا يفعل؟ `Opacity` 0.55 ثم `IgnorePointer`.

لماذا بهذه الطريقة؟ الشفافية تُظهر أنه معطّل، و`IgnorePointer` يمنع أي ضغط حتى لو مرّ.

الأسطر 203–207

```
          child: SettingRow(
            icon: icon,
            iconColor: iconColor,
            tint: tint,
            label: label,
```

ماذا يفعل؟ `SettingRow` بالعنوان والأيقونة.

لماذا بهذه الطريقة؟ نعيد استعمال الصف نفسه.

الأسطر 208–219

```
            trailing: Row(
              mainAxisSize: MainAxisSize.min,
              spacing: 8,
              children: [
                const _SoonChip(),
                RizmaSwitch(
                  value: false,
                  onChanged: (_) {},
                  semanticsLabel: label,
                ),
              ],
            ),
```

ماذا يفعل؟ العنصر الطرفي: `Row` فيها شارة «قريبًا» ثم `RizmaSwitch` مطفأ بدالة فارغة.

لماذا بهذه الطريقة؟ مفتاح مطفأ يطابق شكل التصميم، ودالته فارغة لأنه لا يُضغط أصلًا.

الأسطر 220–225

```
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

الأسطر 227–232

```
class _SoonChip extends StatelessWidget {
  const _SoonChip();

  @override
  Widget build(BuildContext context) {
    return DecoratedBox(
```

ماذا يفعل؟ صنف `_SoonChip` الخاص: `DecoratedBox` عنبري فاتح بزوايا مدوّرة تمامًا.

لماذا بهذه الطريقة؟ لون العنبر للتنبيه اللطيف. خاص بالملف لأنه لا يُستعمل في غيره.

الأسطر 233–246

```
      decoration: BoxDecoration(
        color: AppColors.amberTint,
        borderRadius: BorderRadius.circular(999),
      ),
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 2),
        child: Text(
          ArStrings.accountSoon,
          style: AppTextStyles.chip13.copyWith(color: AppColors.amberDark),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ حشوة 10/2 ونص «قريبًا» بنمط `chip13` عنبري داكن، ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ تباين مقروء على الخلفية الفاتحة.

lib/features/account/widgets/account_sheets.dart — أوراق الاختيار والمعلومات

`lib/features/account/widgets/account_sheets.dart`

الأسطر 1–5

```
import 'package:flutter/material.dart';

import '../../../core/strings/ar.dart';
import '../../../core/theme/app_colors.dart';
import '../../../core/theme/app_text_styles.dart';
```

ماذا يفعل؟ استيراد Material والنصوص والألوان والأنماط.

لماذا بهذه الطريقة؟ ما تحتاجه الأوراق.

الأسطر 7–16

```
Future<T?> _showSheet<T>(BuildContext context, WidgetBuilder builder) {
  return showModalBottomSheet<T>(
    context: context,
    isScrollControlled: true,
    shape: const RoundedRectangleBorder(
      borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
    ),
    builder: builder,
  );
}
```

ماذا يفعل؟ دالة خاصة `_showSheet<T>` : تفتح `showModalBottomSheet` بالشكل الموحّد (قابلة للتمرير، خلفية `ground` ، زوايا علوية 28).

لماذا بهذه الطريقة؟ أوراق حسابي الثلاث تشترك في الشكل فكتبناه مرة. `isScrollControlled` ليتحمّل المحتوى الطويل أو الخط المكبّر. نفس شكل ورقتي الرمز السري.

الأسطر 18–25

```
/// ورقة اختيار واحد من قائمة (مثل حجم الخط). تعيد القيمة المختارة، أو null
/// إن أغلقها الطالب.
Future<T?> showChoiceSheet<T>(
  BuildContext context, {
  required String title,
  required List<(T value, String label)> options,
  required T selected,
}) {
```

ماذا يفعل؟ `showChoiceSheet<T>`: ورقة اختيار واحد. معاملاتها: العنوان وقائمة (قيمة، اسم) والقيمة الحالية.

لماذا بهذه الطريقة؟ عامة بنوع `T` فتخدم حجم الخط الآن وأي اختيار لاحق. سجلات `(T value, String label)` Dart 3 أخف من صنف.

الأسطر 26–28

```
  return _showSheet<T>(
    context,
    (context) => SafeArea(
```

ماذا يفعل؟ نفتح الورقة ونبني `SafeArea`.

لماذا بهذه الطريقة؟ تحمي من حافة الجهاز السفلية.

الأسطر 29–35

```
      child: SingleChildScrollView(
        padding: const EdgeInsetsDirectional.fromSTEB(20, 24, 20, 16),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          spacing: 8,
          children: [
```

ماذا يفعل؟ `SingleChildScrollView` بحشوة 20/24/20/16 و `Column` أصغر ما يمكن بمسافة 8.

لماذا بهذه الطريقة؟ التمرير يحمي من فيض الشاشات القصيرة والخط المكبّر (اكتشفه `layout_stress_test`).

السطر 36

```
            Text(title, style: AppTextStyles.section18),
```

ماذا يفعل؟ عنوان الورقة.

لماذا بهذه الطريقة؟ بنمط `section18`.

الأسطر 37–42

```
            for (final (value, label) in options)
              _ChoiceTile(
                label: label,
                selected: value == selected,
                onTap: () => Navigator.of(context).pop(value),
              ),
```

ماذا يفعل؟ لكل خيار `_ChoiceTile`؛ المحدَّد هو من يساوي `selected`، والضغط يغلق الورقة بقيمته.

لماذا بهذه الطريقة؟ `Navigator.pop(value)` يعيد الاختيار لمن فتح الورقة، و`null` إن أُغلقت بلا اختيار.

الأسطر 43–48

```
          ],
        ),
      ),
    ),
  );
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 50–59

```
class _ChoiceTile extends StatelessWidget {
  const _ChoiceTile({
    required this.label,
    required this.selected,
    required this.onTap,
  });

  final String label;
  final bool selected;
  final VoidCallback onTap;
```

ماذا يفعل؟ صنف `_ChoiceTile` الخاص ومعاملاته (الاسم والاختيار والضغط).

لماذا بهذه الطريقة؟ خاص بهذا الملف.

الأسطر 61–67

```
  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      selected: selected,
      label: label,
      excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` مع `build`: زر محدَّد أم لا باسم الخيار، يستبعد دلالات الأبناء.

لماذا بهذه الطريقة؟ قارئ الشاشة يقول «متوسط، محدَّد».

الأسطر 68–71

```
      child: Material(
        color: selected ? AppColors.tealTint : AppColors.card,
        borderRadius: BorderRadius.circular(18),
        clipBehavior: Clip.antiAlias,
```

ماذا يفعل؟ `Material` بلون `tealTint` للمحدَّد وأبيض لغيره وزوايا 18 وقصّ.

لماذا بهذه الطريقة؟ التمييز اللوني مع علامة ✓ (لا يعتمد على اللون وحده).

الأسطر 72–78

```
        child: InkWell(
          onTap: onTap,
          child: Padding(
            padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
            child: Row(
              children: [
                Expanded(child: Text(label, style: AppTextStyles.body15)),
```

ماذا يفعل؟ `InkWell` وحشوة 16/14 و`Row` والعنوان في `Expanded`.

لماذا بهذه الطريقة؟ العلامة في الطرف.

الأسطر 79–88

```
                if (selected)
                  const Icon(Icons.check_rounded, color: AppColors.teal),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إن كان محدَّدًا نضع علامة ✓ خضراء، ثم إغلاق الأقواس.

لماذا بهذه الطريقة؟ إشارة ثانية غير اللون.

الأسطر 90–96

```
/// صف الكتالوج المتوفّر؛ إن اختلف [contentGrade]. ورقة اختيار الصف (1–12)
/// عنه الصف المختار نذكر للطالب أن محتواه لم يُضف بعد.
Future<int?> showGradeSheet(
  BuildContext context, {
  required int current,
  required int? contentGrade,
}) {
```

ماذا يفعل؟ `showGradeSheet`: ورقة اختيار الصف. معاملاتها: الصف الحالي وصف المحتوى المتوفّر (قد يكون `null`).

لماذا بهذه الطريقة؟ الصف الذي يتوفّر له كتالوج قد يختلف عن المختار: نخبر الطالب بدل أن نعده بمحتوى غير موجود.

الأسطر 97–106

```
  return _showSheet<int>(
    context,
    (context) => SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsetsDirectional.fromSTEB(20, 24, 20, 16),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          spacing: 14,
          children: [
```

ماذا يفعل؟ نفس بنية الورقة (SafeArea وتمرير وعمود) بمسافة 14 بين العناصر.

لماذا بهذه الطريقة؟ نفس الشكل الموحّد.

السطر 107

```
            Text(ArStrings.gradeSheetTitle, style: AppTextStyles.section18),
```

ماذا يفعل؟ عنوان «اختر صفّك».

لماذا بهذه الطريقة؟ من `ArStrings` .

الأسطر 108–119

```
            Wrap(
              spacing: 10,
              runSpacing: 10,
              children: [
                for (var grade = 1; grade <= 12; grade++)
                  _GradeChip(
                    grade: grade,
                    selected: grade == current,
                    onTap: () => Navigator.of(context).pop(grade),
                  ),
              ],
            ),
```

ماذا يفعل؟ `Wrap` بمسافة 10 يضم 12 رقمًا (1–12) كلٌّ في `_GradeChip` والمحدَّد هو الصف الحالي، والضغط يغلق الورقة بالرقم.

لماذا بهذه الطريقة؟ `Wrap` لا شبكة ثابتة: يتكيّف مع عرض الشاشة وحجم الخط، فلا يفيض على 320.

الأسطر 120–124

```
            if (contentGrade != null)
              Text(
                ArStrings.gradeContentSoon(contentGrade),
                style: AppTextStyles.caption13,
              ),
```

ماذا يفعل؟ إن كان صف المحتوى معروفًا نعرض «محتوى هذا الصف لم يُضف بعد...».

لماذا بهذه الطريقة؟ تنبيه صادق: تغيير الصف لا يغيّر الدروس حتى يوجد كتالوج له.

الأسطر 125–130

```
          ],
        ),
      ),
    ),
  );
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 132–141

```
class _GradeChip extends StatelessWidget {
  const _GradeChip({
    required this.grade,
    required this.selected,
    required this.onTap,
  });

  final int grade;
  final bool selected;
  final VoidCallback onTap;
```

ماذا يفعل؟ صنف `_GradeChip` الخاص ومعاملاته (الرقم والاختيار والضغط).

لماذا بهذه الطريقة؟ خاص بهذا الملف.

الأسطر 143–149

```
  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      selected: selected,
      label: ArStrings.gradeSemantics(grade),
      excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` : `build` زر باسم «الصف N» ومحدَّد أم لا.

لماذا بهذه الطريقة؟ قارئ الشاشة يقول «الصف 7» لا «7» فقط.

الأسطر 150–155

```
      child: Material(
        color: selected ? AppColors.teal : AppColors.card,
        shape: StadiumBorder(
          side: BorderSide(color: selected ? AppColors.teal : AppColors.line),
        ),
        clipBehavior: Clip.antiAlias,
```

ماذا يفعل؟ `Material` بشكل كبسولة: أخضر للمحدَّد وأبيض لغيره مع حدّ.

لماذا بهذه الطريقة؟ الشكل الكبسولي مثل بلاطات الصف في الخطوة 1.

الأسطر 156–161

```
        child: InkWell(
          onTap: onTap,
          child: SizedBox(
            width: 56,
            height: 48,
            child: Center(
```

ماذا يفعل؟ `InkWell` و56×48 `SizedBox` والرقم في الوسط.

لماذا بهذه الطريقة؟ 48 ارتفاع مريح للمس.

الأسطر 162–175

```
              child: Text(
                '$grade',
                style: AppTextStyles.gradeNumber24.copyWith(
                  fontSize: 20,
                  color: selected ? AppColors.card : AppColors.ink,
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ الرقم بنمط `gradeNumber24` بحجم 20 ولون أبيض للمحدَّد، ثم إغلاق الأقواس.

لماذا بهذه الطريقة؟ التباين محفوظ في الحالتين.

الأسطر 177–182

```
/// ورقة معلومات نصّية (الخصوصية، عن رزمة).
Future<void> showInfoSheet(
  BuildContext context, {
  required String title,
  required List<String> paragraphs,
}) {
```

ماذا يفعل؟ `showInfoSheet` : ورقة معلومات نصية بعنوان وقائمة فقرات.

لماذا بهذه الطريقة؟ تخدم الخصوصية وعن رزمة بدل شاشتين كاملتين لنص قصير ثابت.

الأسطر 183–191

```
  return _showSheet<void>(
    context,
    (context) => SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsetsDirectional.fromSTEB(20, 24, 20, 16),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.stretch,
          spacing: 10,
```

ماذا يفعل؟ نفس بنية الورقة بمسافة 10.

لماذا بهذه الطريقة؟ نفس الشكل الموحّد.

الأسطر 192–194

```
          children: [
            Text(title, style: AppTextStyles.section18),
            for (final p in paragraphs) Text(p, style: AppTextStyles.body15),
```

ماذا يفعل؟ العنوان ثم نص لكل فقرة بنمط `body15`.

لماذا بهذه الطريقة؟ `for` داخل القائمة.

الأسطر 195–204

```
            TextButton(
              onPressed: () => Navigator.of(context).pop(),
              child: const Text(ArStrings.close),
            ),
          ],
        ),
      ),
    ),
  );
}
```

ماذا يفعل؟ زر نصّي «إغلاق» يغلق الورقة، ثم إغلاق الأقواس.

لماذا بهذه الطريقة؟ يُغلق أيضًا بالسحب أو الضغط خارجها؛ الزر لمن لا يعرف ذلك.